

ProjectOne Engineering Handbook

Chapter 3 — TypeScript Standards

3.1 Objective

TypeScript is used to maximize correctness, readability and maintainability. Every type should communicate intent clearly and prevent invalid states whenever possible.

3.2 General Principles

Enable strict mode. Avoid 'any'. Prefer explicit types for public APIs while allowing inference for simple local variables. Favor immutable data where practical.

3.3 Naming

Interfaces, types and enums use PascalCase. Variables and functions use camelCase. Constants use UPPER_SNAKE_CASE only for true constants.

3.4 Interfaces vs Types

Prefer interfaces for extensible object contracts. Use type aliases for unions, mapped types, utility types and complex compositions.

3.5 Functions

Keep functions focused on a single responsibility. Limit parameters by grouping related values into objects. Avoid hidden side effects.

3.6 Error Handling

Never ignore exceptions. Use typed error objects where possible. Surface user-friendly messages while preserving detailed logs internally.

3.7 Async Code

Always use async/await instead of chained promises unless there is a measurable reason. Await all asynchronous operations explicitly.

3.8 DTOs and Validation

Every external input must pass validation before entering business logic. DTOs define contracts between layers and should remain stable.

3.9 Imports

Use absolute imports when configured. Remove unused imports. Avoid circular dependencies. Public modules expose a single entry point.

3.10 Anti-Patterns

Avoid any, excessive type assertions, deeply nested generics, duplicated types, magic strings and business logic inside utility files.

3.11 Code Review Checklist

Verify type safety, readability, naming consistency, error handling, testability and absence of unnecessary complexity before merging.

Chapter Summary

TypeScript should prevent bugs before runtime. Strong typing is considered part of the architecture, not an optional convenience.